基于 Data Thinker 技术的大规模实时网络日志分析

Large-Scale Real-Time Network Log Analysis Based on Data Thinker

瞿庆海* 顾磷† 陈文涛† 马志强† 张欣源†‡

*上海交通大学 †宁波数方信息技术有限公司 ‡加州大学圣地亚哥分校

摘要

日志分析作为数据中心系统和在线服务运维中重要的技术手段，常用于事件追溯、性能优化、入侵检测、用户行为统计等任务中。传统的日志分析手段，通常在单个或少数计算机上通过文件读写和文本解析命令组合完成，其并行程度有限。随着数据量的增长，传统手段的性能瓶颈日益突出。我们设计和实现了一个并行全文检索计算框架 Greppy，使得若干节点在同一命名空间下可对 TB 级文本日志数据进行关键字检索并达到秒级响应时间，同时具有接近线性的扩展能力。Greppy 使用了 Data Thinker 技术提供内存中计算能力，并融合内外存以提高可扩展性和性价比。通过性能测量和比较，我们验证了 Greppy 系统在数据量和节点数同步增长时能保持常数数据检索时间，而随着节点数增加可获得接近线性的加速比。

Abstract

As an important instrument in the operations of data centers and online services, log data analysis is widely used to trace events, optimize performance, detect intrusions and collect statistics of user activities. Traditionally, the data analysis is operated on one or a small number of computers using file I/O and text mining tools with limited parallelism. In many systems, such techniques have reached their performance limit because of the rapid growth of the data size. We have designed and implemented Greppy, a parallelized log data analysis system, which is able to search TBs of data within seconds. Greppy scales almost linearly to the number of nodes, and leverages the in-memory computing capability of Data Thinker to expedite data processing. Moreover, it combines the memory and disk space to enhance scalability and performance-cost ratio. Performance evaluation shows that Greppy maintains constant execution time when the data and compute nodes increase simultaneously, and achieves near-linear speedup when the number of nodes increases.

关键词: 日志分析，分布式系统，大数据，搜索引擎，内存中计算

Log analysis, distributed system, big data, search engine, in-memory computing

中图法分类号 TP319

1. 简介

随着信息系统和互联网应用的高速发展，越来越多的计算系统以分布式系统的形式在线运行。为了支持对系统的测控、对事件的追溯以及对程序错误的排查，计算系统在运行过程中记录大量运行状态和事件信息，这类信息往往以文件方式存放在规定的文件系统位置，并常常被称为系统日志。常见的日志包括计算事件记录、用户网页浏览记录、计算中间结果、网络报文统计等。通过对日志分析可以实现如对用户行为统计分析、事件回溯等和异常入侵检测等功能。

瞿庆海，男，1976，工学硕士，邮编 200240，Email：qqh@sjtu.edu.cn

顾磷 lingu@shufangkeji.com 陈文涛 xwcwt@shufangkeji.com

马志强 zma@d-thinker.org 张欣源 zhangxinyuan1995@gmail.com

随着计算系统能力的发展，日志数据增长极快，许多系统每天产生数十 GB 甚至数 TB 的日志数据，并保存最近时间段日志数据进行监测和查询。随着磁盘介质单位成本下降、大型分布式文件系统的开源化和键值存储的广泛应用，许多系统已经可以保存若干 TB 日志数据，而国际互联网公司常常会积累和保存若干 PB 的日志数据。

然而，对于海量日志数据的处理和分析依然是一个技术挑战。使用传统的 awk、grep 等搜索和解析工具如在数据量小于 10GB 时可以在合理时间内得到结果。当日志数据集达到百 GB 至 TB 级别，即使辅以大容量文件缓存，传统分析工具和方法依然会产生较长延时。搜索时间往往需要数十分钟至数小时。随着日志数据不断增大，日志分析的瓶颈日益突显，设计一套高效的大数据日志分析技术和系统已经称为一个紧迫需求。

提高日志分析能力的关键在于增加日志数据读取和分析的并行度。Log Insight 等系统对日志的汇集和分析设计了灵活的方案，但是其分析的并行度并不高。谷歌的 Sawzall 系统可以达到较高并行度[1]，然而该技术主要针对离线分析设计，难以有效支持低延时和交互式工作，而且其系统运行需要谷歌基础体系支撑，运行成本较高。

我们设计了一种新型海量数据检索系统 Greppy。该系统基于新型大数据内存计算技术 Data Thinker 及 Puc 语言[2]， 可将众多计算机的大量计算资源集成为一个具有统一数据空间、高并发 I/O 和大规模并行的计算系统。Data Thinker 的一个特点是将内存与外存统一管理，由系统自动进行数据交换和缓冲，这使得大数据处理的成本大大下降。本文主要论述搜索工具 Greppy，同时简要比对 Greppy 和传统搜索工具在 10-230GB 之间数据搜索时的检索时间以及自身的线性扩展能力。

2. 相关工作

传统的日志分析手段通常是在单机上进行关键字搜索时，无法充分利用硬件资源，导致速度十分缓慢。基于 Data Thinker 技术的内存计算搜索工具 Greppy，将所有可调配的硬件资源分割成不同的节点，并选取一个作为主节点向其他节点分配适量的搜索任务，从而充分利用硬件资源，达到较快的搜索速度。Greppy 搜索工具还可以做到弹性利用硬件资源，对于新增的硬件资源仅需简单配置即可用作新的节点，进而增强对数据的处理能力并加快检索速度。因此，相比较 Hadoop[3]及传统的数据业务提供商的产品，Greppy 系统在同样的成本和可靠性指标下，可以获得多达 10 倍的性能。

在硬件资源分配方面，Greppy 与 Log Insight 工具相似，都能够利用新增的硬件资源，高效地扩展数据处理能力。Log Insight 同样是通过扩建节点来实现系统的可扩展性，有助于更快速地完成日志搜索任务。

谷歌公司发布的 Sawzall 是一门专门用于海量数据分析的语言[1]。Sawzall 语言同样使用了并行计算框架，以达到较高的处理性能。 在实现方面，Sawzall 是通过解释器将其转化为相对应的 MapReduce[4]任务。虽然 Sawzall 的语言部分是公开的而且语法明晰，但其性能依赖并受制于其核心并行计算部分，即 MapReduce 框架的能力，尤其是对内存中计算支持有限。

在网络日志分析领域，Analysis Farm 探索了使用 NoSQL 进行日志分析[5]，并取得良好效果。Greppy 系统并不使用特定数据库，而是对文本日志数据直接搜索。

3. Greppy 搜索引擎概述

Greppy 系统由若干节点组成，每个节点兼有数据存储与计算功能，形成一个松散耦合的分布式系统，并可以扩展至上百节点以支持超大规模日志分析。Greppy 系统在所有计算节点中选定一个节点作为主节点，其他均为从节点。主节点通过调度算法对从节点进行任务分配和信息反

馈的收集。Greppy 系统在进行检索之前通过一个预处理过程将待检索数据按照一定的配置分发到各个节点，从宏观上将各个节点抽象成一个具有较强处理能力的“大计算机”，同时对不同的节点采用统一的数据命名空间，使得系统在操作数据的增加、删除上更加灵活。

3.1 Greppy 搜索系统框架

Greppy 搜索系统由服务端(GSE)和客户端(GC)两部分组成，GC 和 GSE 之间通过“Genota”协议进行数据信息的传输和任务的调度，Genota 的详细信息如图 3-1 所示。GC 首先向 GSE 发送信息获取当前搜索数据源的相关信息，信息包括文件位置及大小。GC 对文件信息进一步处理后如果待搜索数据源状态正常则向 GSE 发送待搜索字符串及搜索标志位，如果发现异常则中止操作。GSE 在接受到搜索标志位和待搜索字符串后，从 Portal（主节点）端发送搜索指令到各个数据节点启动搜索，搜索结束后 GSE 向 GC 返回搜索结束标志位以及结果获取方式的相关信息，之后 GC 开始获取搜索结果并对结果根据应用需求进行相关操作。

大部分 Greppy 系统模块由 Puc 语言编制[5]，并运行于 Data Thinker 系统之上，其余部分逻辑由 Shell 和 Python 脚本实现。

3.2 数据处理能力

Greppy 系统在处理能力上可以随节点数的增加线性增长，理论最大支持 16EB(16,000PB)数据的检索，对新增加的节点经过简单的认证配置后主节点就可以对该节点进行调度。节点可以是一台服务器，也可以是一台服务器的部分硬件资源，组成形式灵活，所需硬件成本较低。

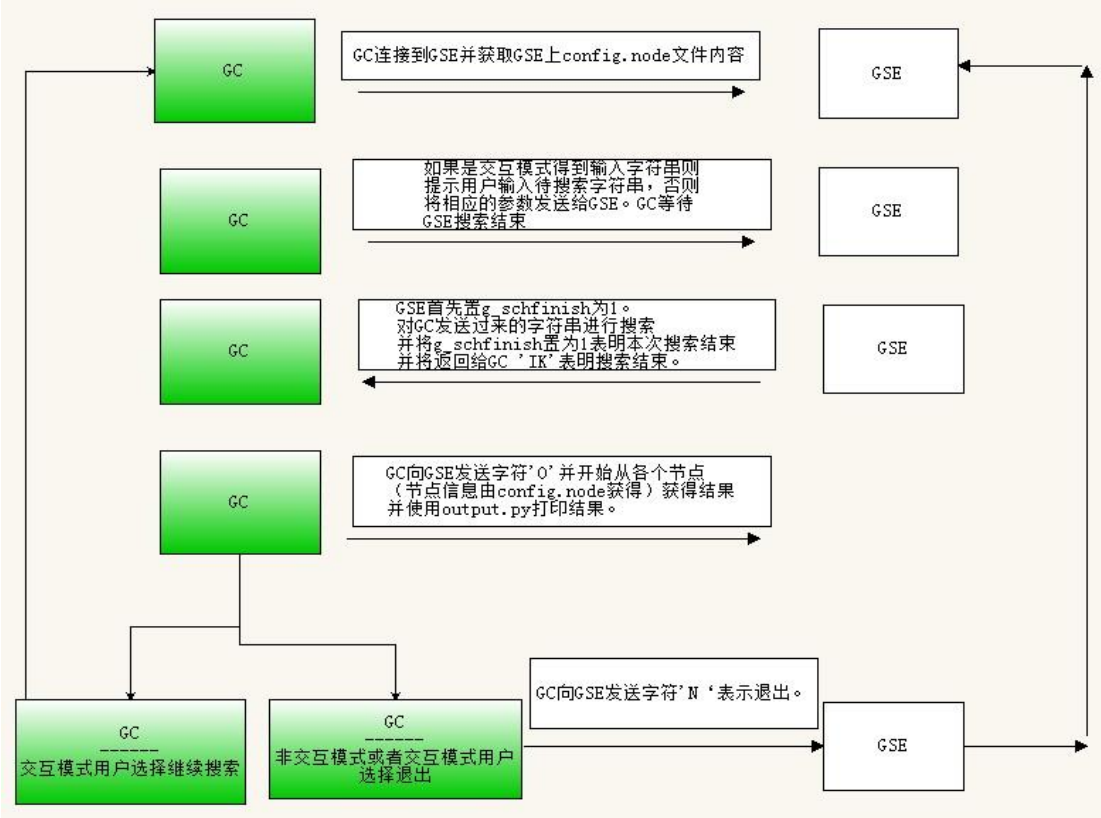


图 3-1 Genota 协议

在数据处理性能上，Greppy 系统从三个方面增强日志分析的能力。首先，Greppy 通过 Data Thinker 的内存中计算功能，极大加速数据处理速度，使得数据读取和计算可以在较短时间内完成，实现对 TB 级数据在秒级时间的全数据检索。其次，Greppy 的架构可以线性扩展，通过增加计算节点在数据量增大时保持低延时搜索性能，在系统硬件支持范围内实现对大数据集的快速检

索。最后，Greppy 将内、外存统一调度，使得数据量超过总内存量时系统能够自动、优雅地将溢出数据存放于外存中进行处理。这种内外存统一调度不但增强了系统的可扩展性和鲁棒性，也大大提高了系统的性价比和实用性。

4. Greppy 系统性能测试

为了验证 Greppy 系统的性能，我们在不同数据量和配置条件下对 Greppy 系统的性能进行评测，并与传统文本搜索工具进行对比。

4.1 Greppy 与传统搜索工具比较

图 4-1 显示在相同配置的服务器上对不同数据进行文本搜索的执行时间对比。从图中可以看出 Greppy 相对于传统搜索工具具有明显优势，使用传统搜索工具 grep 随着数据的集的增长其所需时间呈超线性增长，从 230GB 数据中得到搜索结果需要 2500s，而相同数据集下使用 Greppy 只需要 5S。该结果显示 Greppy 底层架构 Data Thinker 充分利用了硬件的多核、大内存的优势。

4.2 可扩展性

在可扩展性实验中，我们在 Greppy 系统可将所有测试数据均读入内存的条件下，通过增加节点数依次对 200GB 数据的测试并统计数据检索时间，结果如图 4-2 所示。在单节点时处理 200GB 数据所需时间为 35s，在节点数增加至 2 时所需时间为 20s，节点数增至 8 时所需时间为 5s，从中可以看出 Greppy 搜索引擎随着节点数的增加处理性能上大致呈线性增长。

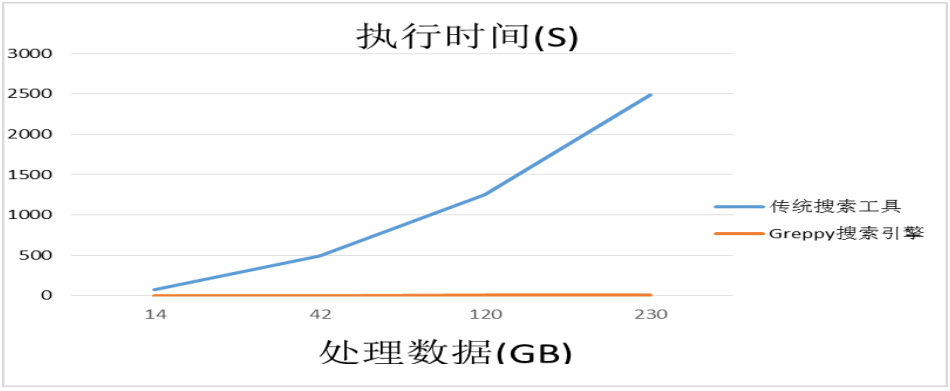


图 4-1 传统工具与并行计算框架检索对比

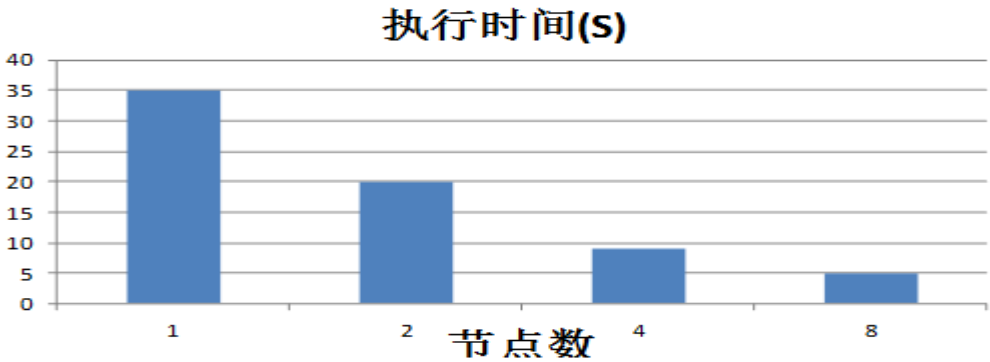


图 4-2 不同节点检索 200GB 数据所需时间

5 结论及展望

基于 Data Thinker 技术的 Greppy 系统在充分利用硬件资源的情况下能够快速地对 TB 及以上的数据进行检索，从而大大提高日志分析的处理时间及效益，后续工作将对 Greppy 增加正则表达式等功能，使系统能够满足更加复杂的需求。

参考文献

[1] Rob Pike, Sean Dorward, Robert Griesemer and Sean Quinlan. Interpreting the data: parallel analysis with Sawzall. Sci. Program. 13, 4, 277-298, Oct. 2005.

[2] Puc Manual. http://d-thinker.org/doc/puc_manual.html [最近访问：2015/8/15]

[3] Hadoop Introduction. http://hadoop.apache.org [最近访问：2015/8/15]

[4] Jeffrey Dean and Sanjay Ghemawat. MapReduce: a flexible data processing tool. Commun. ACM Jan. 2010.

[5] Jianwen Wei, Yusu Zhao, Kaida Jiang, Rui Xie, and Yaohui Jin. Analysis Farm: A cloud-based scalable aggregation and query platform for network log analysis. The First IEEE Workshop on Data Cloud, 2011.